



SITE SCRUTINEERING

Ten findings across the two TheRacingData deployments. Every one was reproduced; three were demonstrated with working exploit code in a headless browser. The two highest both sit on the live main site, and they compound.

FINDINGS	HIGH SEVERITY	PROVEN BY POC	SECRETS FOUND	TOOLS BROKEN
10	2	3	0	3 of 6

SCOPE

Website-Main and Website-ComingSoon — static Cloudflare Pages / Workers sites, 440 files.

METHOD

Source review, then dynamic testing in headless Chromium against both sites served locally.

NOT COVERED

Live DNS, TLS configuration, Cloudflare zone settings and account access were out of scope.

WHAT WAS FOUND

#	SEVERITY	FINDING	MAIN	COMING-SOON
1	HIGH	Stored XSS in Loom Planner via imported project JSON	Exploitable	Blocked by CSP
2	HIGH	Every per-tool CSP rule missing from <code>_headers</code>	Affected	—
3	MEDIUM	Both projects declare the same Worker name	Affected	Affected
4	MEDIUM	No CSP on marketing pages; stated reason fails under test	Affected	Affected
5	MEDIUM	Deployed <code>public/tools/</code> stale — three tools broken	Affected	—
6	LOW	Bundled page bypasses the SRI checks protecting main	—	Affected
7	LOW	<code>dev-server.js</code> containment is a string prefix match	Affected	Affected
8	LOW	<code>postMessage</code> listener has no origin check	Affected	Affected
9	LOW	Template path resolver reaches prototype properties	Affected	Affected
10	INFO	Transport and isolation headers not set	Affected	Affected

THE FINDINGS

Numbered by severity. Findings 1 and 2 are a pair — the second is what turns the first from a bug into working code execution.

01

HIGH

MAIN SITE

PROVEN

STORED XSS IN THE LOOM PLANNER

`public/tools/loom-planner/` renders its state through `innerHTML` with unescaped interpolation, and `importJSON()` accepts a project file with no schema validation. It checks that one top-level key is an array, then hands the whole object to `setState()`:

```
// js/wiring.js:624
const data = JSON.parse(content);
if (data.connectors && Array.isArray(data.connectors)) setState(data);
```

From there the file's contents reach eleven sinks unescaped — colour fields dropped straight into `style` attributes, numeric fields assumed to be numbers, ids written into `data-` attributes:

```
js/wiring.js:871, 1027 style="background:${group.colour};color:${T.PAPER};"
js/wiring.js:1224 style="border-left-color:${group.colour};"
js/canbus.js:534 <span class="${dirClass}">${msg.direction}</span>
js/canbus.js:581 data-sig-edit="${sig.id}"
js/utils.js:92, 118 value="${val}" value="${displayText}"
```

The `escHtml()` helper applied elsewhere is itself only correct for element-text context. It escapes `<`, `>` and `&` but not quotes, so it is unsafe inside a quoted attribute even where it *is* used:

```
// js/wiring.js:2161 – and the identical copy at js/canbus.js:817
function escHtml(str) {
  const d = document.createElement('div');
  d.textContent = str;
  return d.innerHTML; // escapes < > & – but NOT " or '
}
```

OBSERVED — CRAFTED WIRING_LOOM.JSON, IMPORTED THROUGH THE REAL UI

```
window.__XSS_WIREGROUP set : true
window.__XSS_CONNGROUP set : true
injected <img src=x> nodes : 3
XSS EXECUTED : true
```

The CAN bus tab falls the same way through `#can-import`, via `msg.direction` and `sig.id`. And because `app.js` autosaves to `localStorage` on `beforeunload`, one import is enough — the payload re-fires on every later visit with no further user action:

OBSERVED — AFTER IMPORT, THEN A PLAIN PAGE LOAD

```
localStorage keys : [ 'wiring_planner_state', 'canbus_planner_state' ]
XSS re-fires on plain page load (no re-import) : true
```

A wiring loom or CAN configuration is exactly the kind of file a team passes around. One opened file buys an attacker persistent script execution on `theracingdata.com`, with access to everything else same-origin — including the other tools' saved sessions.

FIX

1. **Escape for the context.** Use an attribute-safe escaper for anything inside a quoted attribute, or set values with `setAttribute`, `textContent` and `el.style.x =` rather than building HTML strings.
2. **Validate on import.** Colours against `/^[0-9a-f]{3,8}$/i` or a name allowlist; numeric fields through `Number()`; ids regenerated locally rather than trusted from the file.
3. **Ship the CSP** (finding 2) — it contains the damage even when a sink is missed.

02 HIGH MAIN SITE PROVEN

EVERY PER-TOOL CSP RULE IS MISSING

The coming-soon site carries hardened per-tool policies. The main site's `_headers` is byte-identical up to line 27 and then simply stops — the entire tools section is gone:

```
$ diff Website-Main/public/_headers Website-ComingSoon/public/_headers
27a28,56
> /tools/report-builder/*
> Content-Security-Policy: default-src 'none'; script-src 'self'; ...
> /tools/loom-planner/*
> Content-Security-Policy: default-src 'none'; script-src 'self'; ...
> /tools/report-builder/templates/*
> Cache-Control: public, max-age=31536000, immutable
```

This is the difference between an injection bug and code execution. Replaying the identical finding-1 payload with the coming-soon loom-planner policy applied:

OBSERVED — SAME PAYLOAD, COMING-SOON CSP APPLIED

```
injected <img src=x> nodes : 3      (HTML injection still happens)
XSS EXECUTED                : false
CSP violations logged        : 3
    e.g. Refused to execute inline event handler because it violates ...
```

Same bug, same payload. The policy stops it dead.

The other four tools have no policy on *either* site. `_headers` explains that a new tool "gets no CSP until it is added here; that fails open rather than shipping a blank page." That trade-off costs nothing here — I loaded each unprotected tool under the strictest policy already in use and all four render with zero violations:

TOOL	DOM NODES	CSP VIOLATIONS
<code>/tools/run-plan/</code>	82	none
<code>/tools/fuel-stint/</code>	488	none
<code>/tools/event-pack/</code>	150	none
<code>/tools/load-budget/</code>	168	none

FIX — RESTORE THE SECTION, THEN EXTEND IT

```
/tools/run-plan/*
/tools/fuel-stint/*
/tools/event-pack/*
/tools/load-budget/*
Content-Security-Policy: default-src 'none'; script-src 'self'; style-src 'self';
    font-src 'self'; img-src 'self' data;; connect-src 'self'; form-action 'none';
    base-uri 'none'; frame-ancestors 'none'
```

Consider inverting the default while you are in the file: a broad `/tools/*` policy with named exceptions fails closed instead of open.

03 **MEDIUM** BOTH SITES

BOTH SITES DEPLOY TO THE SAME WORKER

```
// Website-Main/wrangler.jsonc AND Website-ComingSoon/wrangler.jsonc – identical
{ "name": "yellow-cake-de51", "compatibility_date": "2026-08-20", ... }
```

Two separate sites, one deployment target. Whichever is published last silently replaces the other, with no warning at deploy time. The coming-soon site is the more hardened of the two and the main site is the one carrying the live tools, so either direction is a bad outcome.

FIX

Give each project its own `name`.

04

MEDIUM

BOTH SITES

PROVEN

NO CSP ON THE MARKETING PAGES

`_headers` documents this as deliberate: *"No Content-Security-Policy is set on purpose. [...] A CSP without `'unsafe-eval'` renders a blank page."* I tested that claim against the actual pages. It does not hold.

TEST	PAGE	RESULT
Baseline, no CSP	<code>/index.html</code>	renders — 286 nodes
Candidate with <code>'unsafe-eval'</code>	<code>/index.html</code>	renders — 286 nodes, 0 violations
Candidate without <code>'unsafe-eval'</code>	<code>/index.html</code>	still renders — 271 nodes, degrades to props only
Candidate + <code>'unsafe-inline'</code> + <code>blob:</code>	<code>/tools/index.html</code>	renders — 196 nodes
Candidate	<code>/404.html</code>	renders — 48 nodes

The page never went blank. The inline `<script type="text/x-dc">` block is inert to the browser — unknown MIME type — so it needs no `script-src` allowance at all. Only `support.js`, `image-slot.js` and the `unpkg` bundles do.

A policy with `'unsafe-eval'` is weaker than one without, but far stronger than nothing: it still blocks injected external script sources, `<object>` and `<embed>`, `<base>` hijacking and form exfiltration.

A related dependency. `support.js` loads React, ReactDOM and Babel from `unpkg.com` at page load. During testing, with `unpkg` unreachable, the marketing pages rendered **completely blank** — zero nodes, zero characters of body text, no fallback. The SRI hashes are pinned and I verified all three against the live upstream files; they match exactly, so integrity is sound. Availability rests entirely on a third-party CDN.

FIX — TESTED POLICY FOR /INDEX.HTML, /404.HTML AND THE DESIGN-EXPORT PAGES

```
default-src 'none'; script-src 'self' 'unsafe-eval' https://unpkg.com;
style-src 'self' 'unsafe-inline' https://fonts.googleapis.com;
font-src 'self' https://fonts.gstatic.com data:; img-src 'self' data: blob:;
connect-src 'self' https://unpkg.com; object-src 'none'; base-uri 'none';
form-action 'none'; frame-ancestors 'none'
```

/tools/index.html additionally needs 'unsafe-inline' and blob: in script-src. Separately, consider vendoring the three CDN files, as the tools already do for Archivo and docx-templates.

05 MEDIUM MAIN SITE

THE DEPLOYED TOOLS DIRECTORY IS STALE

OBSERVED — NODE TOOLS/BUILD-TOOLS.MJS --CHECK

STALE	tools/_shared/report-engine/resolve.js
STALE	tools/_shared/report-engine/store.js
STALE	tools/_shared/report-engine/values.js
STALE	tools/build-tools.mjs
ORPHAN	tools/fuel-stint/README.md
ORPHAN	tools/fuel-stint/js/state.js
ORPHAN	tools/fuel-stint/js/ui.js

Three modules that _shared/report-engine/docx.js imports are absent from the deploy directory. Loading the affected pages confirms real breakage — half of the tool suite:

PAGE	HTTP FAILURES	RENDERED
/tools/report-builder/	404 store.js	broken
/tools/run-plan/	404 store.js, resolve.js	38 nodes (82 on coming-soon)
/tools/event-pack/	404 resolve.js, store.js	33 nodes (150 on coming-soon)

The coming-soon deploy directory is out of sync in the other direction — 10 stale files, 10 orphaned.

FIX

- 1. Run node tools/build-tools.mjs before packaging, and wire --check into CI so a stale deploy directory fails the build.
- 2. Move _tools/build-tools.mjs to tools/, where a copy already lives. It currently sits inside the tree that is copied verbatim to public/tools/, so once the build runs the build script itself is published at /tools/build-tools.mjs.

06 LOW COMING-SOON

THE BUNDLED PAGE BYPASSES SRI

support.js protects its CDN loads with subresource integrity — but the check is skipped whenever a resource override is present:

```
// public/support.js:1149
function cdnScriptFor(url, sri) {
  const res = window.__resources;
  const v = res ? res[url] : void 0;
  return typeof v === "string" && v
    ? { src: v } // no integrity on this branch
    : { src: url, integrity: sri };
}
```

The coming-soon `index.html` is a self-extracting bundle that decodes React and ReactDOM from embedded base64, turns them into blob URLs, and injects exactly that map:

```
<script type="__bundler/ext_resources">
[{"id":"https://unpkg.com/react@18.3.1/umd/react.production.min.js",
  "uuid":"38ef6850-b0d7-4866-8497-38cc4b82229b"}, ...]
```

The blobs are same-origin and delivered over TLS, so practical risk is low. But the bundled bytes execute with no verification of any kind, and the SRI hashes that protect the main site do not apply here. If the bundling step is ever compromised, nothing catches it.

FIX

Verify bundled bytes against the pinned hashes before creating the blob URL.

07 **LOW** BOTH SITES **PROVEN**

DEV-SERVER CONTAINMENT IS A PREFIX MATCH

```
const ROOT = path.join(__dirname, 'public');
let file = path.join(ROOT, rel);
if (!file.startsWith(ROOT)) { res.writeHead(403).end('forbidden'); return; }
```

`path.join` normalises `..`, so classic traversal is blocked. But `startsWith(ROOT)` also accepts any sibling directory whose name begins with `public` — `public.bak`, `public-secret`, `public_old`:

OBSERVED

```
$ curl 'http://localhost:8788/..%2fpublic-secret/leak.txt'
```

SECRET-OUTSIDE-ROOT

```
$ curl -o /dev/null -w '%{http_code}\n' 'http://localhost:8788/..%2f..%2fetc%2fpasswd'
403
```

Local preview tooling only, bound to `localhost:8788` — but `public.bak/` is precisely the sort of directory that accumulates next to a deploy folder.

FIX

```
const rp = path.relative(ROOT, file);
if (rp.startsWith('.') || path.isAbsolute(rp)) { res.writeHead(403).end('forbidden'); return; }
```

MESSAGE LISTENER HAS NO ORIGIN CHECK

`public/support.js:1407` accepts messages from any origin. Reachable impact is small — the theme value is validated against `"light"` and `"dark"`, and `X-Frame-Options: DENY` stops the site being framed at all.

```
window.addEventListener("message", (e) => {
  const type = e.data && e.data.type;      // no e.origin check
  if (type === "__dc_theme") { ... }
  if (!designDocMode || type !== "__dc_probe") return;
  postDesignMode(designDocMode);
});
```

FIX

Check `e.origin` against an allowlist, and replace the `"*"` targets on the two outbound `postMessage` calls (lines 1387 and 1858) with a concrete origin.

RESOLVER REACHES PROTOTYPE PROPERTIES

`_shared/report-engine/resolve.js` is a genuinely good piece of design: it replaces docx-templates' JavaScript evaluation with a strict grammar, so a template can never execute code and the page needs no `'unsafe-eval'`. One gap — `lookup()` walks any dotted path, and `__proto__`, `constructor` and `prototype` all match the identifier pattern:

```
const PATH = /^\$?[A-Za-z_][\w]*(?:\.[A-Za-z_][\w]*)*$/;
```

A template containing `{constructor.constructor}` resolves to the `Function` constructor. The result is only stringified into the document, so this is an information-disclosure edge rather than code execution.

FIX

Reject those three segment names explicitly, and prefer `Object.hasOwn(value, key)` in the walk.

TRANSPORT AND ISOLATION HEADERS

Currently set on `/*`, on both sites: `X-Content-Type-Options`, `Referrer-Policy`, `X-Frame-Options`, `Permissions-Policy`. Worth adding:

```
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
```


Confirm whether HSTS is already applied at the Cloudflare zone level before adding it here.

`Permissions-Policy` could also deny `payment=()`, `usb=()` and `serial=()`.

03 · CLEAN

WHAT HOLDS UP

Worth recording, because several of these are things sites of this kind usually get wrong.

No secrets anywhere

Scans for AWS keys, GitHub and Slack tokens, PEM blocks and credential-shaped assignments came back clean across both trees. No `.env`, no lockfiles, no npm dependency surface.

SRI hashes are correct

All three pinned `sha384` values match the live upstream files byte for byte. The pinning is real, not decorative.

The docx resolver is a real mitigation

Finding 9 notwithstanding. `noSandbox` is not enabled, and the custom `runJs` avoids both the `iframe` and `new Function()` paths in docx-templates.

The templates are clean

No `vbaProject.bin`, no OLE objects and no `TargetMode="External"` references across all 20 `.docx` templates. No active content in any SVG.

Loom Planner is the outlier, not the pattern

`run-plan`, `event-pack`, `fuel-stint` and `load-budget` contain zero uses of `innerHTML`, `insertAdjacentHTML`, `document.write`, `eval` or `new Function`.

Build and link hygiene

No `Invoke-Expression`, `shell=True` or `pickle.load` in the build and dev scripts. No `target="_blank"` without `rel="noopener"`, no `http://` subresources, no open-redirect patterns.

Tool data stays local

The tools persist to `localStorage` and export to file. Nothing is transmitted — which is also why finding 1 lands where it does.

04 · SEQUENCE

ORDER OF WORK

Ordered by effect per unit of effort, not by severity alone.

1	Restore and extend the tool CSP rules One file, biggest effect. It also contains finding 1 while that fix is in progress. — finding 2
2	Fix the Loom Planner escaping and validate on import Both tabs, all eleven sinks, plus the shared <code>escHtml</code> helper. — finding 1
3	Give the two projects distinct Worker names A two-character change that removes a silent whole-site overwrite. — finding 3

4

Re-run the tools build and add `--check` to CI

Restores three broken tools and stops the next stale deploy. — finding 5

5

Add the tested marketing-page CSP

The policy in finding 4 is verified against every affected page. — finding 4

6

Findings 6 to 10 as maintenance

SRI on bundled resources, dev-server containment, origin checks, resolver hardening, transport headers.

THERACINGDATA · SECURITY REVIEW

Findings were reproduced against the supplied archives served locally; live DNS, TLS and Cloudflare zone configuration were not in scope. Full write-up with line references is committed at `security/2026-08-24-website-security-review.md`.